

On the Effectiveness of Deep Learning Techniques for Malware Detection and their Resilience to Obfuscation



Author:
Henry Williams

September 2025

Supervisor:
Jeff Mitchell

Outline

1. Introduction
2. Malware
3. Detection
4. Results
5. Conclusion

Introduction

Research Questions

RQ1: Are modern natural language processing techniques, such as language models and pre-training effective at identifying malware?

RQ2: Does pre-training improve the accuracy and reliability of malware detection models based on techniques from natural language processing?

RQ3: To what extent are static malware detection models reliant on obfuscation as an indicator of malicious programs?

Malware

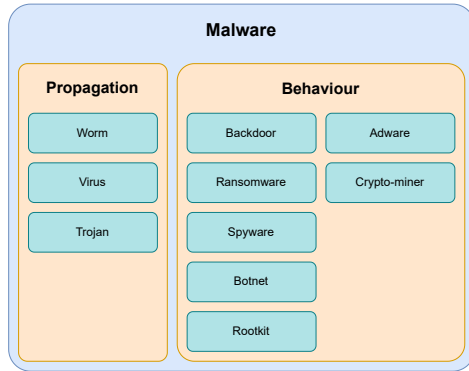


Figure: The different types of malware, based on their behavior and method of spread

Malware

Definition

Malware — Software that has a harmful effect on the typical operation of a host system.¹

- Provides unauthorised access
- Allows extraction or modification of information
- Prevents authorised users from accessing it

¹Kramer and Bradfield 2010.

Malware

Obfuscation

```
def encrypt(path, crypt):
    with open(path, "rb") as f:
        d = f.read()

        d = crypt.encrypt(d)

    with open(path, "wb") as f:
        f.write(d)

def lock_dir(directory: str, crypt):
    files = os.listdir(directory)

    for file in files:
        file = os.path.join(directory, file)
        if os.path.isfile(file):
            print(f'Encrypting {file}')
            encrypt(file, crypt)
        else:
            lock_dir(file, crypt)

if __name__ == "__main__":
    crypt = Encryptor("f4k3-k3y-4-t3st1ng")
    lock_dir("./", crypt)
    print("Your files have been locked! Please pay 1XMR to regain access")
```

(a) Before

```
IIIIIIII=(lambda getattr:(((lambda
IIII,IIII:setattr(__builtins__,IIII,IIII)))(IIII,IIII)) for IIII,IIII
in getattr.items()}));II=chr(114)+chr(101);IIr='[^a-zA-
Z0-9]';II=chr(115)+chr(117)+chr(98);IIIIIIIIII, IIIIIIIIII,
IIIIIIIIII,IIIIIIIIIIII = __import__, getattr, bytes,exec

__import__["sys"].setrecursionlimit(1099511615);IIIIIIIIIIII{IIIIIIIIII
II{IIIIIIIIIIII{IIIIIIIIIIII.fromhex('7a6c6962').decode()},
IIIIIIIIIIII.fromhex('6465636f6d7872657373').decode()}
{IIIIIIIIIIII.fromhex('789ced5dd<remaining payload removed for clarity>
1133feff0'.replace("\n", ""))}.decode())
```

(b) After

Figure: Ransomware shown before, and after obfuscation

Malware Detection

Types

- **Static** — File structure
 - Metadata
 - Byte sequences
 - Cryptographic hash
- **Dynamic** — Behavioral characteristics
 - API call sequences
 - System modifications
 - Network communication
- **Hybrid** — Mix of static and dynamic features

Our Approach

Dataset

- Lack of suitable datasets
 - EMBER² & BODMAS³ do not distribute executables
 - Others do not distribute benign executables
- Malicious samples from VirusShare
 - Large scale database of malware seen in the wild
 - > 100 million samples
- Benign samples from my personal system
 - Video games
 - Developer tools
 - Custom software
 - Pre-installed programs (system executables, internet browser, etc.)
 - Verified benign with windows defender
 - Approx. 5000 in total

²Anderson and Roth 2018.

³Yang et al. 2021.

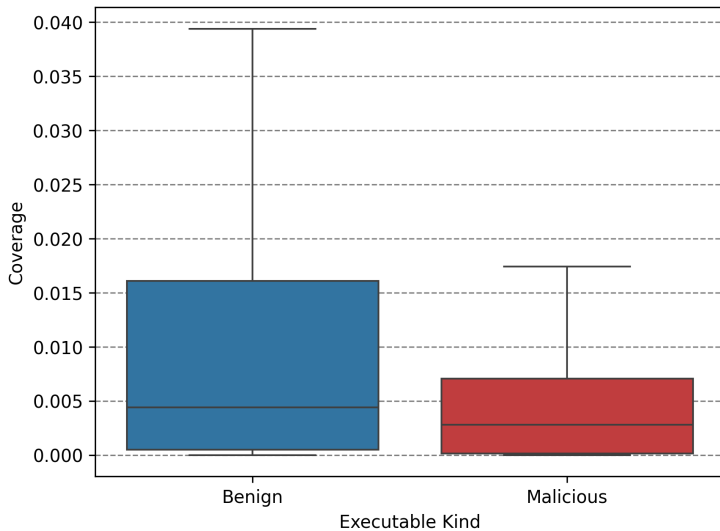
Our Approach

Dataset — Obfuscation

- Obfuscation is more pervasive in malicious software
- Analysis coverage — ratio of the amount of code analysed to its total amount in a program
- Coverage is typically lower in obfuscated programs
- *Do static approaches learn to identify obfuscation as an indicator of malware?*

Our Approach

Dataset — Obfuscation



Our Approach

Models

- Sequence-based
 - Identify malicious instruction sequences, aggregate sequence level classification across the file.
 - BERT architecture⁴
 - Pretraining?
- Image-based
 - Convert programs to images
 - Identify malicious image-representations
 - EfficientNet B0 architecture⁵

⁴Devlin et al. 2019.

⁵Tan and Le 2020.

Our Approach

Sequence Classification

ret lea mov add add lea mov add add nop

Figure: A sequence of opcodes used by our model

- Hyperparameter sweep to identify best architecture configuration
- Does pre-training on Masked Language Modelling help?
- Can we only train the classification head for this problem?

Our Approach

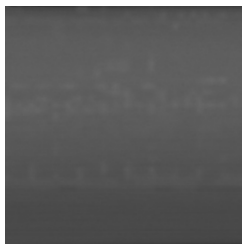
Sequence Classification — Architecture

Table: BERT sequence classification model parameters

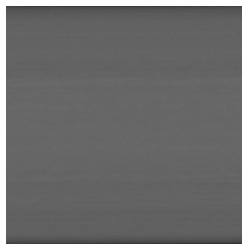
Parameter	Value
Sequence Length	32
Hidden Layer Size	512
Hidden Layers	7
Attention Heads	4
Intermediate Layer Size	2048
Activation Function	GELU
Hidden Dropout Rate	0.01
Attention Dropout Rate	0.001

Our Approach

Image Classification



(a) Benign



(b) Malicious

Figure: Average image representation of each class

- Malicious programs are often much smaller than benign programs
- Benign programs are resized more than malicious programs
- Does this model use scaling artifacts to distinguish between malicious and benign?

Our Approach

Image Classification — Size Distribution

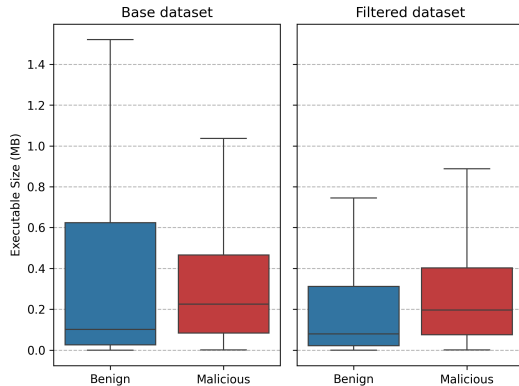


Figure: Comparison between the program size distribution for each class in our base dataset, and the filtered dataset

Results

Baseline

Table: Performance of baseline methods for malware detection using the import tables of executables

Algorithm	Feature	Accuracy	F1 Score	Precision	Recall
Support Vector Classifier	DLL	0.8859	0.8856	0.8856	0.8856
	Function	0.9248	0.9248	0.9266	0.9262
Logistic Regression	DLL	0.8682	0.8682	0.8687	0.8695
	Function	0.9438	0.9438	0.9442	0.9447
Decision Tree	DLL	0.8877	0.8876	0.8876	0.8885
	Function	0.9406	0.9406	0.9415	0.9417

Results

Sequence Classification

Table: Performance of our RoBERTa model for opcode sequence classification and malware detection

Model	Sequence Level				File Level			
	Accuracy	F1-Score	Precision	Recall	Accuracy	F1-Score	Precision	Recall
From Scratch	0.8756	0.8742	0.8798	0.8756	0.8976	0.8971	0.9095	0.8989
From pretrained	0.9010	0.9002	0.9042	0.9010	0.9452	0.9452	0.9479	0.9458
From pretrained (w/ Frozen Attention Weights)	0.8302	0.8265	0.8409	0.8302	0.8076	0.8037	0.8393	0.8098

Results

Image Classification

Table: Performance of the image classification models for malware detection

Dataset	Accuracy	F1-Score	Precision	Recall
Full	0.9316	0.9315	0.9314	0.9322
Filtered	0.9306	0.9306	0.9309	0.9306

Results

Sequence Classification — Obfuscation

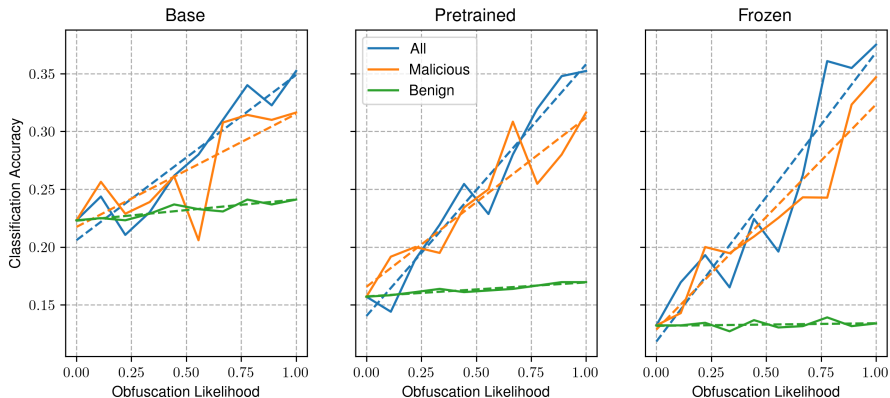


Figure: How obfuscation impacts the file level classification accuracy of our Sequence classification models.

Results

Image Classification — Obfuscation

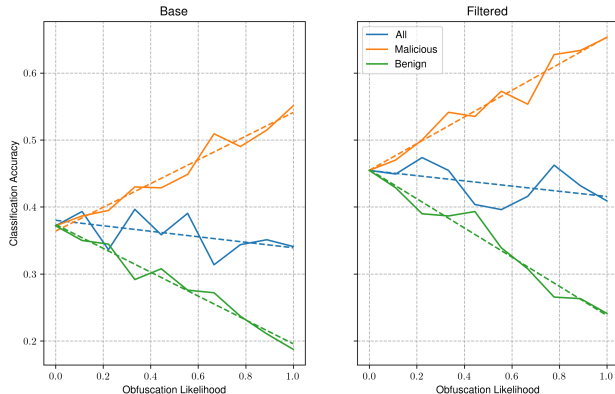


Figure: How obfuscation impacts the file level classification accuracy of our image classification models.

Conclusion

Findings

- Sequence classification model offers best performance overall
- Image classification model does not surpass our baseline, and is highly reliant on obfuscation as an indicator of malware.
- Sequence classification model is also reliant on obfuscation but to a lesser extent

Conclusion

Limitations

- Dataset limitations
 - Coverage analysis in secondary dataset
 - Significant performance disparity between primary and secondary dataset
- Choice of opcode sequences
 - Full disassembly for basic blocks?

Questions?

Malware

Impact

- WannaCry
 - Infected 200,000 systems
 - \$4 billion
 - Crippled the National Health Service
- Stuxnet
 - Infected 60% of all computers in Iran
 - Significantly impacted their ability to enrich nuclear material
 - First known example of a “Cyber-Weapon”

Opcode Extraction

Algorithm 1 Generate and extract disassembly from an executable

Input: Path to executable P

Output: Sequence of disassembled instructions of P

```
1:  $L \leftarrow$  empty list
2: Load  $P$  into radare2
3: Perform full analysis                                ▷ R2 Command: aaa
4: if Instruction set of  $P$  is not x86 then
5:     return
6: end if
7:  $S \leftarrow$  list of sections in  $P$                     ▷ R2 Command: iS
8:  $S_x \leftarrow$  filter  $S$  for executable sections
9: for all  $s \in S_x$  do
10:     $o \leftarrow$  section offset of  $s$ 
11:     $n \leftarrow$  section size of  $s$ 
12:     $D \leftarrow$  disassemble bytes from  $o$  to  $o + n$     ▷ R2 Command: pda  $o @ o + n$ 
13:     $D' \leftarrow$  filtered  $D$  to remove padding bytes
14:    Append  $D'$  to  $L$ 
15: end for
16: return  $L$ 
```

Base Model

F1 Score

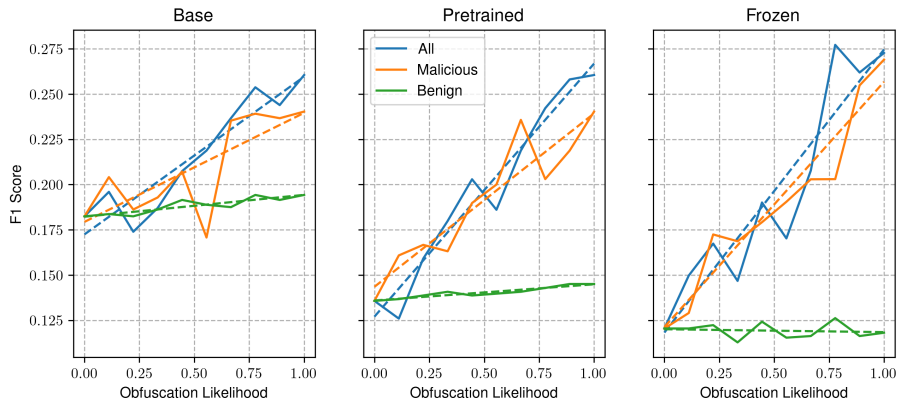


Figure: File Level F1 score

Base Model

Sequence Accuracy

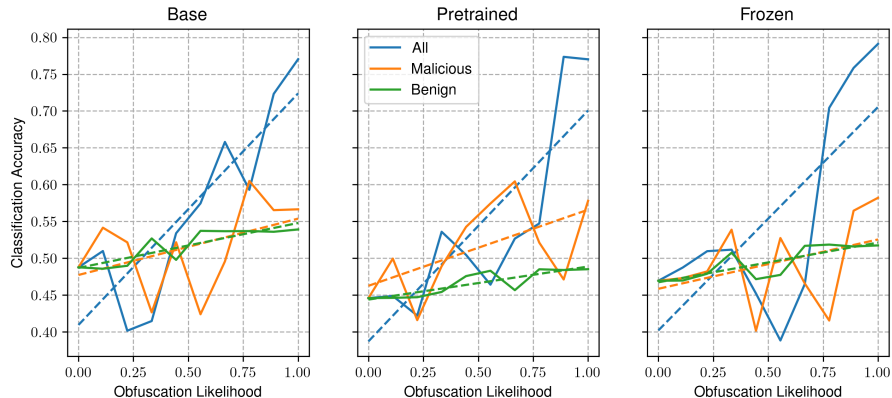


Figure: Sequence-level performance

Base Model

Certainty

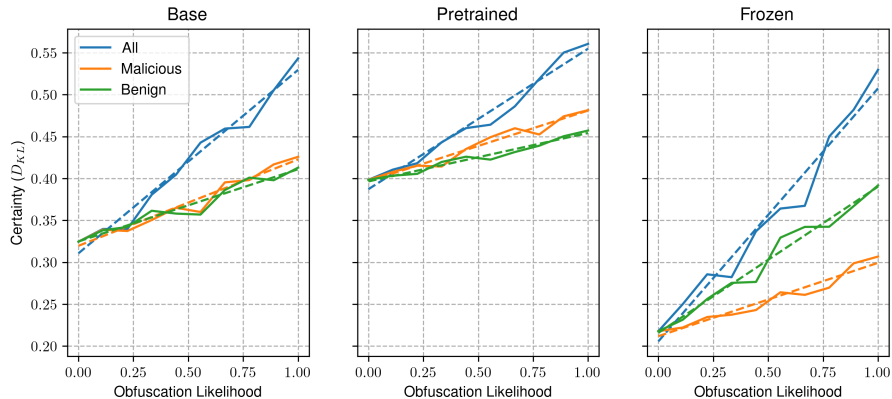
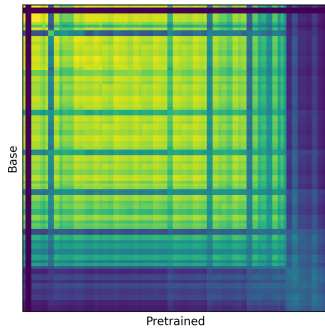


Figure: Confidence

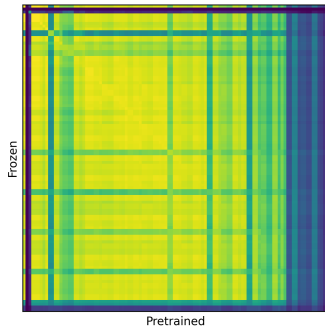
Base Model

RSA of Pretrained and Non-Pretrained Models



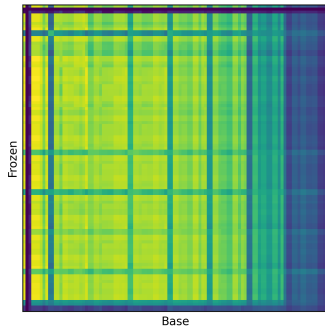
Base Model

RSA of Pretrained and Frozen Pretrained Models



Base Model

RSA of Frozen Pretrained and Non-Pretrained Models



Obfuscated Benign Model

Accuracy

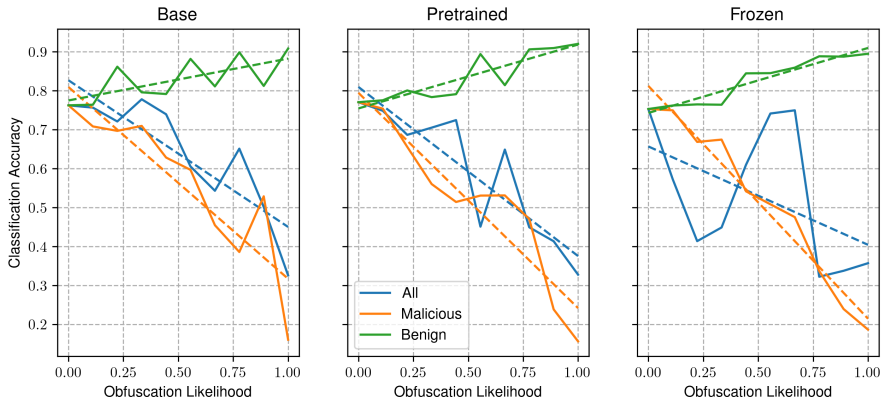


Figure: File-level performance

Obfuscated Benign Model

F1 Score

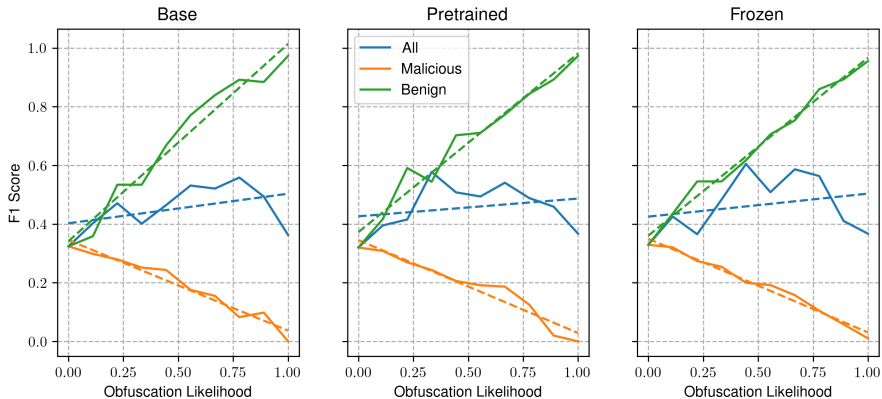


Figure: File Level F1 score

Obfuscated Benign Model

Sequence Accuracy

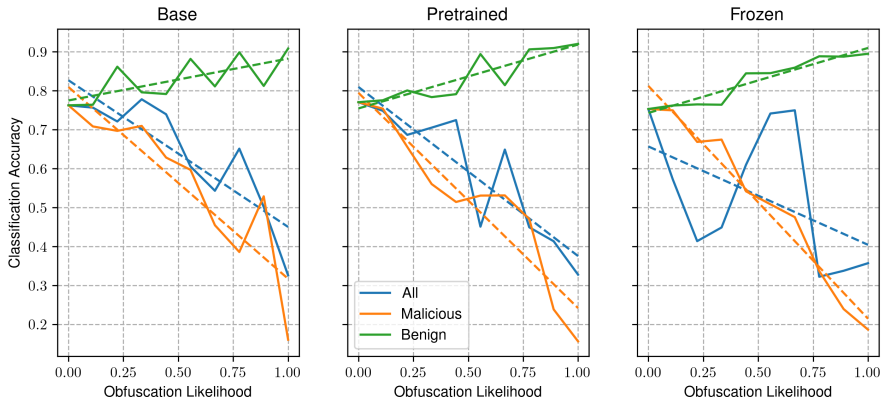


Figure: Sequence-level performance

Obfuscated Benign Model

Certainty

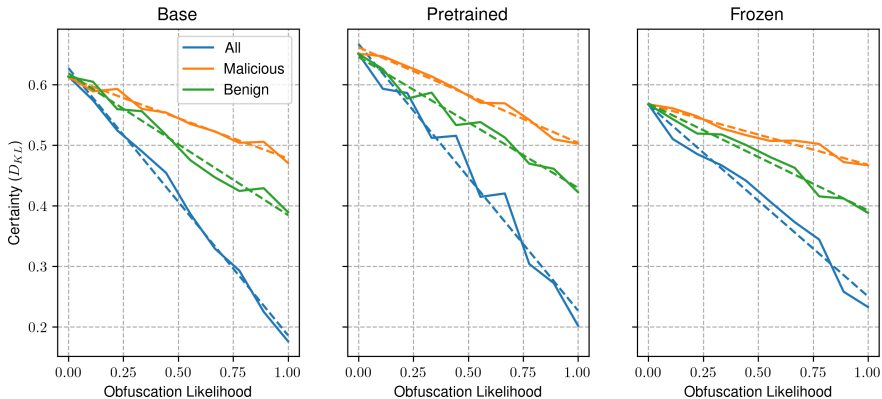
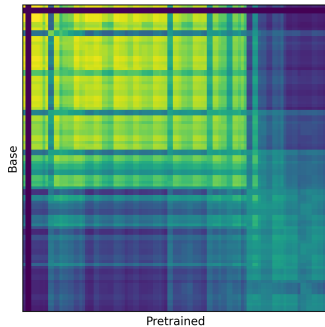


Figure: Confidence

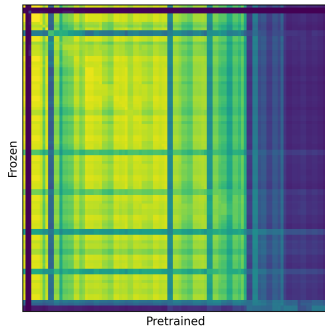
Obfuscated Benign Model

RSA of Pretrained and Non-Pretrained Models



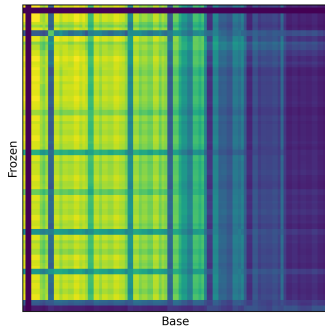
Obfuscated Benign Model

RSA of Pretrained and Frozen Pretrained Models



Obfuscated Benign Model

RSA of Frozen Pretrained and Non-Pretrained Models



Partially Obfuscated Benign Model

Accuracy

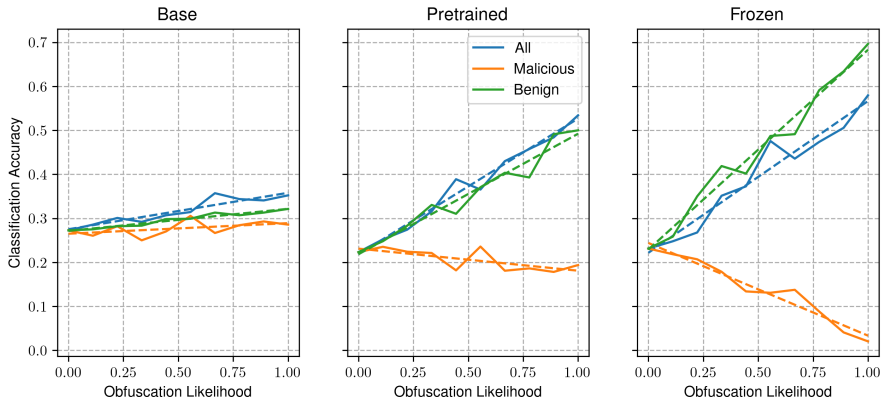


Figure: File Level F1 score

Partially Obfuscated Benign Model

F1 Score

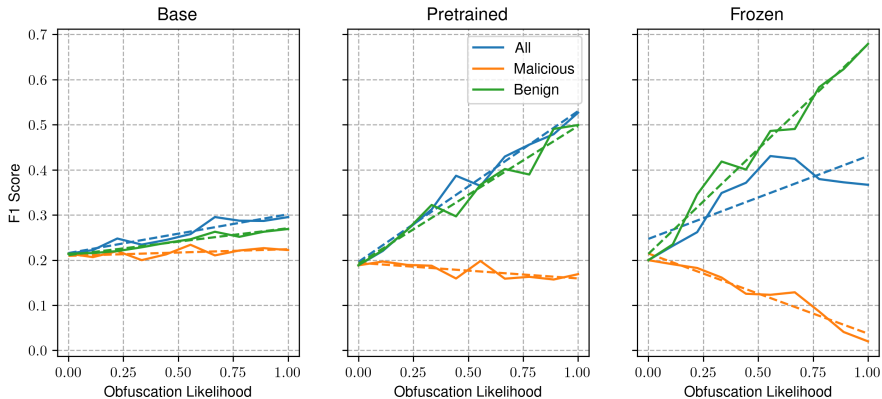


Figure: File Level F1 score

Partially Obfuscated Benign Model

Sequence Accuracy

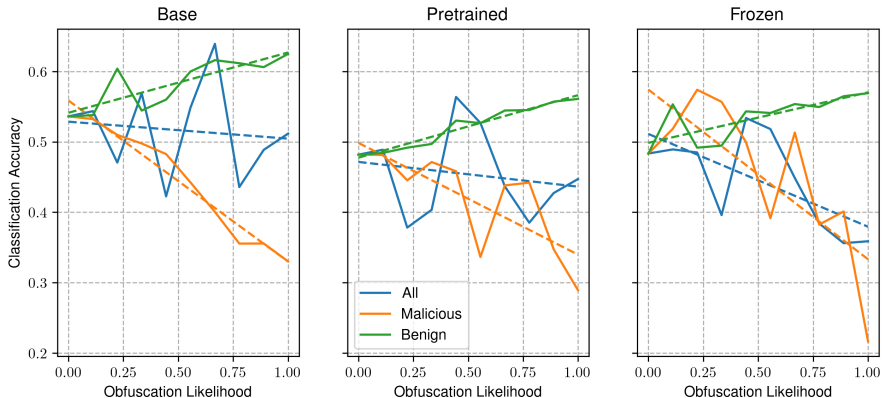


Figure: Sequence-level performance

Partially Obfuscated Benign Model

Certainty

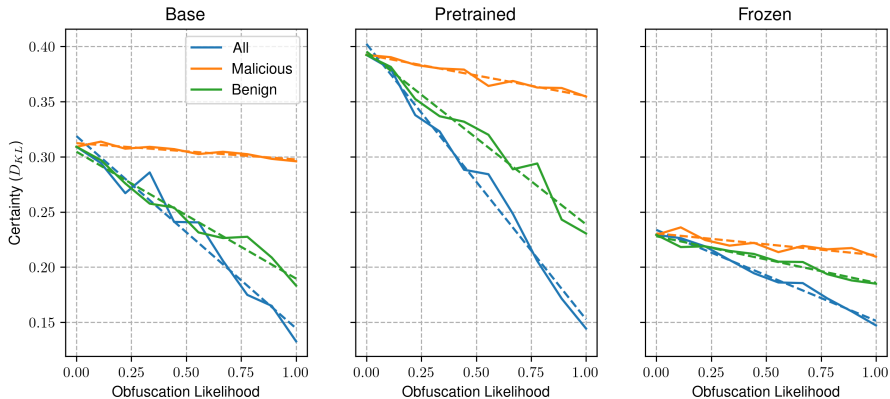
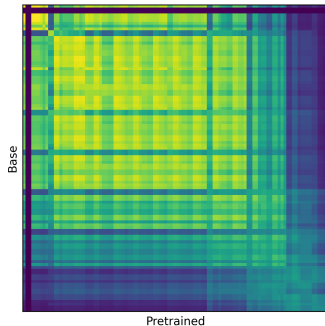


Figure: Confidence

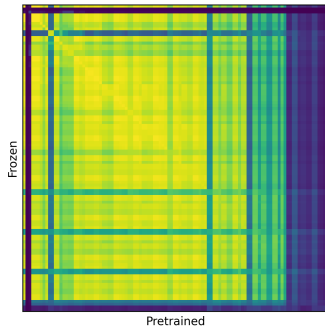
Partially Obfuscated Benign Model

RSA of Pretrained and Non-Pretrained Models



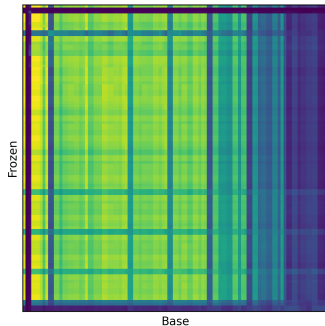
Partially Obfuscated Benign Model

RSA of Pretrained and Frozen Pretrained Models



Partially Obfuscated Benign Model

RSA of Frozen Pretrained and Non-Pretrained Models



Obfuscation Metric

Entropy

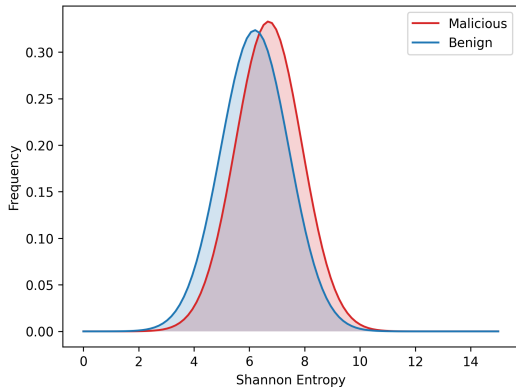
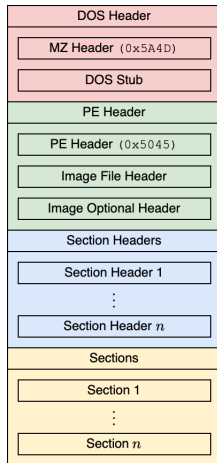


Figure: Distribution of entropy for malicious and benign samples

Windows PE File Structure



Secondary Dataset — Analysis Coverage

